Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Représentation binaire des entiers relatifs, algorithme du complément à deux, soustraction unifiée et détection du dépassement de capacité (Overflow)

Synthèse exécutive : Ce cours détaille les principes matériels et algorithmiques qui permettent aux ordinateurs de représenter et de manipuler les nombres entiers relatifs (positifs et négatifs) en mémoire binaire. En l'absence de composants électroniques capables de stocker un symbole abstrait '+' ou '-', le matériel s'appuie sur la convention du complément à deux sur N bits fixe. Cette méthode résout les deux problèmes majeurs du codage 'Signe et Magnitude' : elle élimine le double zéro (+0 et -0) et permet d'utiliser le même circuit électronique d'addition au sein de l'Unité Arithmétique et Logique (ALU) pour effectuer à la fois les additions et les soustractions (A - B = A + (-B)). La maîtrise de la procédure de conversion en 3 étapes, le calcul de la plage de valeurs dissymétrique [-2^(N-1), +2^(N-1)-1] et la distinction fondamentale entre la retenue sortante (Carry out) et le dépassement de capacité (Overflow) constituent des compétences indispensables pour éviter les erreurs de programmation bas niveau et réussir les évaluations.

1. Problématique matérielle et justification du Complément à Deux

Au niveau le plus élémentaire de l'architecture d'un processeur, la mémoire vive et les registres sont formés de circuits électroniques bistables (bascules). Ces composants ne reconnaissent que deux états physiques distincts : l'absence de tension (0 Volt, représentée par le 0 logique) et la présence de tension (Vcc, représentée par le 1 logique). Il est physiquement impossible de graver un signe '+' ou '-' dans un transistor. Pour coder des entiers négatifs, la première approche historique a été le codage 'Signe et Magnitude', où le bit le plus à gauche sert d'indicateur de signe et les bits suivants portent la valeur absolue. Cette convention souffre de deux graves défauts : elle génère un double zéro (+0 et -0), ce qui gaspille une valeur binaire utile et alourdit la logique de comparaison, et elle exige la présence de deux circuits électroniques distincts au sein de l'Unité Arithmétique et Logique (ALU) : un additionneur et un soustracteur. Le complément à deux élimine complètement ces contraintes. En attribuant au zéro un codage unique (tous les bits à 0) et en transformant toute soustraction A - B en une addition A + (-B), l'ALU réutilise son circuit d'addition binaire standard pour toutes les opérations arithmétiques fondamentales.

- Absence de signe matériel : Les bascules et transistors ne stockent que deux états logiques : 0 (0 Volt) ou 1 (Vcc). Aucun composant ne peut stocker directement '+' ou '-'.
- Limites du codage 'Signe et Magnitude' : Présence de deux représentations pour zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison binaire et impose de construire des circuits matériels séparés pour l'addition et la soustraction.
- Unification par le Complément à Deux : Le zéro possède un codage unique (0000...0) et la soustraction A - B s'effectue sous forme d'une addition A + (-B) sur le même circuit additionneur de l'ALU.

Cas concret enseignant : *La métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur affiche 0000 km et que vous effectuez un kilomètre en marche arrière, le compteur repasse mécaniquement à 9999. En binaire sur 8 bits, retrancher 1 à 00000000_2 donne 11111111_2, ce qui constitue la définition exacte de -1 en complément à deux.*

2. Algorithme de Conversion et Plage de Valeurs sur N bits

Pour représenter un entier négatif -X en complément à deux, il faut obligatoirement définir au préalable la taille N du mot mémoire (ex: 8, 16, 32 ou 64 bits). L'algorithme de conversion se déroule en trois étapes strictes et séquentielles. Tout d'abord, on écrit la valeur absolue X en binaire pur et on la cadre rigoureusement sur N bits en complétant par des 0 à gauche jusqu'à atteindre exactement N chiffres binaires. Ensuite, on effectue le complément à un en inversant la totalité des bits (les 0 deviennent 1 et les 1 deviennent 0). Enfin, on ajoute 1 à ce résultat binaire pour obtenir la représentation en complément à deux. Le bit le plus à gauche (MSB) vaut obligatoirement 1

pour un nombre négatif. En raison de l'inclusion du zéro (0000...0) dans le sous-ensemble dont le MSB vaut 0, la plage des nombres représentables sur N bits est dissymétrique : il y a un nombre négatif de plus que de nombres positifs (ex: sur 8 bits, de -128 à +127).

- Algorithme des 3 étapes : 1. Écrire la valeur absolue X sur N bits complets ; 2. Inverser chaque bit (Complément à 1) ; 3. Ajouter 1 au résultat (Complément à 2).
- Rôle du bit MSB (Most Significant Bit) : Le bit de poids fort (le plus à gauche) sert d'indicateur de signe (0 = positif ou nul, 1 = négatif).
- Plage de valeurs dissymétrique : Sur N bits, les valeurs représentables s'étendent de -2^(N-1) à +(2^(N-1) - 1). Sur 8 bits, cela correspond à [-128, +127].

Cas concret enseignant : *Avertissement crucial du professeur pour les partiels : il est IMPÉRATIF de fixer la taille N bits (ex: 8 bits) et d'ajouter les zéros à gauche AVANT d'effectuer l'inversion des bits. Inverser la séquence '1100' (+12 sur 4 bits) sans écrire les zéros de cadrage '00001100' donne '0011' au lieu de '11110011', ce qui conduit à une réponse totalement fausse en examen !*

3. Arithmétique, Retenue (Carry Out) et Détection du Dépassement de Capacité (Overflow)

Les registres d'un processeur ayant une largeur fixe, les opérations arithmétiques peuvent générer deux drapeaux (flags) distincts : le Carry Out (C) et l'Overflow (OV). Lors d'une addition en complément à deux, la retenue sortante située à la position N+1 (au-delà du bit de poids fort) est ignorée car l'architecture matérielle effectue ses calculs en arithmétique modulaire de période 2^N. Il ne faut pas la confondre avec l'Overflow. L'Overflow signale une erreur d'amplitude : le résultat mathématique réel ne peut pas être encodé sur les N bits disponibles. La règle d'évaluation formelle pour détecter un Overflow lors de l'addition de deux entiers signés est la suivante : un Overflow se produit si et seulement si on additionne deux nombres de même signe (MSB identiques) et que le résultat possède un signe inverse (MSB altéré). Si l'on additionne deux nombres de signes différents, un Overflow est physiquement et mathématiquement impossible.

- Soustraction unifiée : Calculée directement sous la forme A - B = A + (-B).
- Retenue sortante (Carry Out) : Retenue générée au-delà du bit MSB, systématiquement ignorée en arithmétique signée en complément à deux car le processeur travaille modulo 2^N.
- Drapeau d'Overflow (OV) : Indique un dépassement de capacité où le résultat sort de la plage autorisée [-2^(N-1), +2^(N-1)-1].
- Règle d'or de l'Overflow : Un Overflow survient si et seulement si l'addition de deux opérandes de même signe (deux positifs ou deux négatifs) produit un résultat de signe opposé.

Cas concret enseignant : *L'anecdote historique du crash d'Ariane 5 (vol 501 en 1996) : une conversion de type d'un nombre flottant 64 bits vers un entier signé 16 bits sans protection contre le dépassement de capacité a généré un Overflow non géré. L'Unité de Référence Inertielle a transmis une valeur corrompue au calculateur de bord, interprétée à tort comme un écart de trajectoire, ce qui a provoqué le braquage brutal des tuyères puis l'autodestruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant) est-elle jetée lors de l'addition en complément à deux ?
Réponse du professeur : L'ALU travaille sur un nombre fixe de N bits déterminé par l'architecture des registres (ex: 8 bits). La retenue qui dépasse le N-ième bit ne peut pas être enregistrée dans le registre de résultat faute d'emplacement physique. D'un point de vue mathématique, le complément à deux repose sur une arithmétique modulaire modulo 2^N : ignorer la retenue sortante de valeur 2^N équivaut exactement à conserver le reste de la division euclidienne par 2^N, ce qui donne la valeur exacte du résultat dans l'anneau des entiers relatifs modélisés.

Question étudiant : Comment l'ordinateur fait-il la différence entre le nombre binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : L'ordinateur ne fait aucune différence au niveau des bits stockés en mémoire : une suite de bits est amorphe et n'a aucune signification intrinsèque. C'est le contexte d'exécution qui définit le sens de la donnée. Le processeur utilise des instructions d'assemblage différentes (ex: `ADD` pour le signé, `ADDU` pour le non-signé) ou applique le type de donnée déclaré par le programmeur dans son code source (ex: `int` vs `unsigned int` en C). Le matériel exécute aveuglément les opérations binaire bas niveau, et ce sont les drapeaux de registre (Carry pour le non-signé, Overflow pour le signé) qui permettent au logiciel d'interpréter correctement le résultat.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler le codage binaire des deux opérandes en complément à deux, poser l'addition binaire correspondante, donner le résultat décimal final et analyser la présence d'un éventuel Overflow.

Résolution pas à pas :
Étape 1 : Conversion de +35 sur 8 bits. 35 = 32 + 2 + 1 = 00100011_2. Étape 2 : Conversion de -75 en complément à deux sur 8 bits. - Valeur absolue (+75) en binaire pur sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion de tous les bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (Représentation de -75). Étape 3 : Pose et calcul de l'addition binaire (+35) + (-75). 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 Étape 4 : Interprétation du résultat binaire 11011000_2. Le bit MSB est à 1, le résultat est négatif. Pour déterminer sa valeur absolue décimale, on réapplique le complément à deux sur 11011000_2 : - Inversion des bits : 00100111_2 - Ajout de 1 : 00100111 + 1 = 00101000_2 = 32 + 8 = 40 décimal. Le mot binaire 11011000_2 représente donc -40 en décimal. Étape 5 : Analyse du drapeau de dépassement de capacité (Overflow). On additionne un nombre positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). Les deux opérandes possédant des signes opposés, l'apparition d'un Overflow est mathématiquement impossible.

Résultat : Résultat binaire : 11011000_2 (égal à -40 décimal). Drapeau d'Overflow OV = 0 (pas de dépassement de capacité). Le calcul 35 - 75 = -40 est exact et valide.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage exacte des entiers signés représentables sur un mot mémoire de 16 bits en complément à deux ?

R : La plage est de -2^(16-1) à +(2^(16-1) - 1), soit de -32 768 à +32 767.

Q : Comment détecte-t-on de façon certaine la présence d'un Overflow lors d'une addition de deux entiers signés ?

R : Un Overflow survient si et seulement si l'on additionne deux opérandes de même signe (deux MSB à 0 ou deux MSB à 1) et que le résultat obtenu possède un signe opposé (MSB différent).

Q : Quel est le résultat de la procédure du complément à deux appliquée au nombre zéro (00000000_2) sur 8 bits ?

R : Inversion des bits : 11111111_2. Addition de 1 : 11111111 + 1 = 100000000_2. Sur 8 bits, la retenue sortante est rejetée, ce qui redonne 00000000_2. Cela prouve l'unicité de la représentation du zéro.